CUET PG

DAILY CLASS NOTES

Computer Science

Operating System

Lecture – 10

Deadlock: Part 2



Deadlock: Part 2

Topics to be Covered

Deadlock: Part 2

1. **Banker's Algorithm**
2. **Safe Sequence Calculation**
3. **Examples and Numerical Problems**

Banker's Algorithm

Definition

- Banker's Algorithm is a **Deadlock Avoidance Algorithm**.
- It is used to **avoid deadlocks** by ensuring the system **always remains in a safe state**.

Working of the Algorithm

(a) Process Admission:

- When a new process enters the system, it **declares its maximum resource requirements**.
- These requirements can occur **anytime during execution**.
- The process is **admitted only if** its maximum needs **do not exceed the total system resources**.
- The system ensures that **all processes can complete in some sequence**, called a **Safe Sequence**.

(b) Resource Request:

- During execution, when a process requests resources (within its declared maximum), The system checks if granting the request will keep it in a **safe state**.
- If yes → **resources are allocated**.
  If not → **process waits**.

Example 1:

- **Safe Sequence Calculation**

  Given:

  Total Resources = **13**

Process	Allocation	Maximum
P_0	5	10
P_1	4	6
P_2	1	2

  Step 1:

  Calculate Need

  Need = Maximum – Allocation

Process	Allocation	Maximum	Need
P_0	5	10	5
P_1	4	6	2
P_2	1	2	1
Total	**10**		

  Step 2:

  Calculate Free Resources

  Total = 13

  Allocated = 10

  Free = 13 – 10 = 3

Step 3:
Find Safe Sequence

1. **Available = 3**
   - P_2 Need = 1 → can execute
   - After P_2 finishes → Available = 3 + 1 (its allocation) = **4**
2. **Available = 4**
   - P_1 Need = 2 → can execute
   - After P_1 finishes → Available = 4 + 4 = **8**
3. **Available = 8**
   - P_0 Need = 5 → can execute
   - After P_0 finishes → Available = 8 + 5 = **13**

Safe Sequence = <P_2, P_1, P_0>

Example 2:

- **Multiple Resource Types (A, B, C)**

Given:
Total resources = **A=10, B=5, C=7**
Free resources = **A=3, B=3, C=2**

Process	Allocation (A B C)	Maximum (A B C)
P_0	0 1 0	7 5 3
P_1	2 0 0	3 2 2
P_2	3 0 2	9 0 2
P_3	2 1 1	2 2 2
P_4	0 0 2	4 3 3

Step 1:
Calculate Need
Need = Maximum – Allocation

Process	Allocation (A B C)	Maximum (A B C)	Need (A B C)
P_0	0 1 0	7 5 3	7 4 3
P_1	2 0 0	3 2 2	1 2 2
P_2	3 0 2	9 0 2	6 0 0
P_3	2 1 1	2 2 2	0 1 1
P_4	0 0 2	4 3 3	4 3 1

Step 2:
Determine Safe Sequence
Initial Available = (3, 3, 2)

1. Check **P_1**: Need (1,2,2) ≤ Available (3,3,2) → Execute
   - New Available = (3,3,2) + (2,0,0) = **(5,3,2)**
2. Check **P_3**: Need (0,1,1) ≤ (5,3,2) → Execute
   - New Available = (5,3,2) + (2,1,1) = **(7,4,3)**
3. Check **P_4**: Need (4,3,1) ≤ (7,4,3) → Execute
   - New Available = (7,4,3) + (0,0,2) = **(7,4,5)**
4. Check **P_0**: Need (7,4,3) ≤ (7,4,5) → Execute
   - New Available = (7,4,5) + (0,1,0) = **(7,5,5)**
5. Check **P_2**: Need (6,0,0) ≤ (7,5,5) → Execute
   - New Available = (7,5,5) + (3,0,2) = **(10,5,7)**

Safe Sequence = <P_1, P_3, P_4, P_0, P_2>

डेडलॉक: भाग 2

शामिल किए जाने वाले विषय

डेडलाक: भाग 2

1. **बैंकर एल्गोरिथम**
2. **सेफ सीक्वेंस कैलकुलेशन**
3. **उदाहरण और संख्यात्मक समस्याएं**

बैंकर एल्गोरिथम

परिभाषा

- बैंकर एल्गोरिदम एक डेडलॉक से बचने का एल्गोरिदम है।
- इसका उपयोग यह सुनिश्चित करके डेडलॉक से बचने के लिए किया जाता है कि सिस्टम हमेशा सुरक्षित स्थिति में रहे।

एल्गोरिदम का कार्यप्रणाली

(a) प्रोसेस एडमिशन:

- जब कोई नया प्रोसेस सिस्टम में आता है, तो वह अपनी अधिकतम रिसोर्स ज़रूरतों की घोषणा करता है।
- ये ज़रूरतें एग्जीक्यूशन के दौरान कभी भी हो सकती हैं।
- प्रोसेस को तभी एडमिट किया जाता है जब उसकी अधिकतम ज़रूरतें कुल सिस्टम रिसोर्स से ज़्यादा न हों।
- सिस्टम यह सुनिश्चित करता है कि सभी प्रोसेस किसी खास क्रम में पूरे हो सकें, जिसे सेफ सीक्वेंस कहा जाता है।

(b) रिसोर्स रिक्वेस्ट:

- एग्जीक्यूशन के दौरान, जब कोई प्रोसेस रिसोर्स की रिक्वेस्ट करता है (अपने बताए गए मैक्सिमम के अंदर),
- सिस्टम चेक करता है कि रिक्वेस्ट देने से सिस्टम सेफ स्टेट में रहेगा या नहीं।
- अगर हाँ → रिसोर्स एलोकेट कर दिए जाते हैं।
  अगर नहीं → प्रोसेस इंतज़ार करता है।

उदाहरण 1:

- **सेफ सीक्वेंस कैलकुलेशन**

  दिया गया:

  कुल रिसोर्स = **13**

प्रोसेस	एलोकेशन	मैक्सिमम
P_0	5	10
P_1	4	6
P_2	1	2

चरण 1:

नीड की गणना करें

नीड = मैक्सिमम – एलोकेशन

प्रोसेस	एलोकेशन	मैक्सिमम	नीड
P_0	5	10	5
P_1	4	6	2
P_2	1	2	1
कुल	**10**		

चरण 2:

फ्री रिसोर्स की गणना करें

कुल = 13

एलोकेटेड = 10

फ्री = 13 – 10 = 3

चरण 3:

सेफ सीक्वेंस ज्ञात करें

1. **अवेलेबल = 3**
   - P_2 नीड = 1 → एग्जीक्यूट हो सकता है
   - P_2 के खत्म होने के बाद → अवेलेबल = 3 + 1 (इसका एलोकेशन) = 4
2. **अवेलेबल = 4**
   - P_1 नीड = 2 → एग्जीक्यूट हो सकता है
   - P_1 के खत्म होने के बाद → अवेलेबल = 4 + 4 = 8
3. **अवेलेबल = 8**
   - P_0 नीड = 5 → एग्जीक्यूट हो सकता है
   - P_0 के खत्म होने के बाद → अवेलेबल = 8 + 5 = 13

सेफ सीक्वेंस = <P_2, P_1, P_0>

उदाहरण 2:

- कई संसाधन/रिसोर्स प्रकार (A, B, C)

दिया गया:

कुल रिसोर्स = **A=10, B=5, C=7**

फ्री रिसोर्स = **A=3, B=3, C=2**

प्रोसेस	**एलोकेशन (A B C)**	**मैक्सिमम (A B C)**
P_0	0 1 0	7 5 3
P_1	2 0 0	3 2 2
P_2	3 0 2	9 0 2
P_3	2 1 1	2 2 2
P_4	0 0 2	4 3 3

चरण 1:

नीड ज्ञात करें

नीड = मैक्सिमम – एलोकेशन

प्रोसेस	**एलोकेशन (A B C)**	**मैक्सिमम (A B C)**	**नीड (A B C)**
P_0	0 1 0	7 5 3	7 4 3
P_1	2 0 0	3 2 2	1 2 2
P_2	3 0 2	9 0 2	6 0 0
P_3	2 1 1	2 2 2	0 1 1
P_4	0 0 2	4 3 3	4 3 1

चरण 2:

सेफ सीक्वेंस निर्धारित करें

प्रारंभिक अवेलेबल = (3, 3, 2)

1. P_1 चेक करें: नीड (1,2,2) ≤ अवेलेबल (3,3,2) → एग्जीक्यूट
   - नया अवेलेबल = (3,3,2) + (2,0,0) = (5,3,2)
2. P_3 चेक करें: नीड (0,1,1) ≤ (5,3,2) → एग्जीक्यूट
   - नया अवेलेबल = (5,3,2) + (2,1,1) = (7,4,3)
3. P_4 चेक करें: नीड (4,3,1) ≤ (7,4,3) → एग्जीक्यूट
   - नया अवेलेबल = (7,4,3) + (0,0,2) = (7,4,5)
4. P_0 चेक करें: नीड (7,4,3) ≤ (7,4,5) → एग्जीक्यूट
   - नया अवेलेबल = (7,4,5) + (0,1,0) = (7,5,5)
5. P_2 चेक करें: नीड (6,0,0) ≤ (7,5,5) → एग्जीक्यूट
   - नया अवेलेबल = (7,5,5) + (3,0,2) = (10,5,7)

सेफ सीक्वेंस = <P_1, P_3, P_4, P_0, P_2>



PW Web/App: https://smart.link/7wwosivoicgd4
Library- https://smart.link/sdfez8ejd80if